Dr. Dobb's Digest June 2009

Break Up and Interleave Work to Keep Threads Responsive

Breaking up is hard to do, but interleaving can be even subtler

By Herb Sutter

Herb Sutter is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at www.gotw.ca.

In a recent article, we covered reasons why threads should strive to make their data private and communicate using asynchronous messages.[1] Each thread that needs to receive input should have an inbound message queue and organize its work around an event-driven message pump mainline that services the queue requests, idealized as follows:

```
// An idealized thread mainline
//
do {
   message = queue.pop() // get the message
                         // (might wait)
   message->run();    // and execute it
} while( !done );         // check for exit
```

But what happens when this thread must remain responsive to new incoming messages that have to be handled quickly, even when we're in the middle of servicing an earlier lower-priority message that may take a long time to process?

If all the messages must be handled on this same thread, then we have a problem. Fortunately, we also have two good solutions, both of which follow the same basic strategy: somehow break apart the large piece of work to allow the thread to perform other work in between, interleaved between the chunks of the large item. Let's consider the two major ways to implement that interleaving, and their respective tradeoffs in the areas of fairness and performance.

Example: Breaking Up a Potentially Long-Running Operation

Consider this potentially expensive message we might be asked to execute:

```
// A simplified message type to accomplish some
// long operation
//
class LongOperation : public Message {
public:
   void run() {
      LongHelper helper = GetHelper();
```

```
// issue: what if this loop could take a long time?
  for( int i = 0; i < items.size(); ++i ) {
    helper->render( items[i] );
  }
  helper->print();
 }
}
```

This thread may be a background worker that runs all the work we want to get off the GUI thread (see [1]). Alternatively, in cases where it is impossible to obey the good hygiene of getting all significant work off the GUI thread (for example, because for some reason the work may need to happen on the GUI thread itself for legacy or OS-specific reasons), this thread may be the GUI itself. Whatever the case, what matters is that to remain responsive to other messages we need to break up **LongOperation.run** into smaller pieces and interleave them with the processing of other messages.

Option 1:Use Continuations

The first way to tackle the problem is to use continuations. A continuation is just a way to talk about "the rest of the work that's left to do." It includes capturing the state of any local or intermediate variables that we're in the middle of using, so that when we resume the computation we can correctly pick up again where we left off.

Example 1(a) shows one way to implement a continuation style:

```
// Example 1(a): Using a continuation style
//
class LongOperation : public Message {
 int start;
   LongHelper helper;
public:
 LongOperation( int start_ = 0,
      LongHelper helper_ = nullptr )
        : start(start_), helper(helper_) { }
  void run() {
 if( helper == nullptr
            // if first time through, get helper
       helper = GetHelper();
      int i = 0;
 // do just another chunk's worth
      for( ; i < ChunkSize && start+i < items.size();
           ++i ) {
       helper->render( items[ start+i ] );
  }
 if( start+i < items.size() )
       // if not done, launch a continuation
    queue.push(LongOperation(start+i, helper));
  else
           // if last time through, finish up
      helper->print();
```

```
  }
}
```

The first **LongOperation**object executes only a suitably-sized chunk of its loop. To ensure that the remainder of the work gets done, it creates a new **LongOperation** object (the continuation) that stores the current intermediate state, including the helper local variable and the loop index, and pushes the continuation on the message queue. (For simplicity this code assumes **LongOperation** has direct access to queue; in practice you would provide an indirection such as a callback to decouple the message type from a specific queue.)

A good way to think about this is that we're "saving our stack frame" off in a separate object on the heap. The overhead we're incurring for each continuation is the cost of copying the local variables, one allocation (and subsequent destruction) for the continuation message, and one extra pair of enqueue/dequeue operations. This approach has the major advantage of fairness. It's fair to the waiting messages, because each continuation gets pushed onto the end of the queue and so all messages currently waiting will be processed before we do another chunk of the longer work. More importantly, it's fair to **LongOperation** itself, because any other new messages that arrive after the continuation is enqueued, during the time while we're draining the queue, will stay in line behind the continuation.

This fairness can also be a disadvantage, however, if we'd actually like to execute most of the messages in order no matter how long they take, and only enable interleaved "early" execution for certain high-priority messages. Some of that can be accomplished using a priority queue as the message queue, but in general this kind of flexibility will be easier to accomplish under Option 2, where we opt for a different set of tradeoffs.

Beware State Changes When Interleaving

Note that there is a subtle but important semantic issue that we have to be aware of that wasn't possible in the noninterleaved version of the code.

The issue is that whenever the code cedes control to allow other messages to be executed, it must be aware that the thread's state can be changed by that other code that executed on this thread in the meantime. When the code resumes, it cannot in general assume that any data that is private to the thread, including thread-local variables, has remained unchanged across the interruption.

In Example 1(a), consider: What happens if another message changes the size or contents of the **items** collection? If our operation is trying to process a consistent snapshot of the collection's state, we may need to save even more off to the side by taking a snapshot of the collection and doing all of our work against the snapshot, so that we maintain the semantics of doing our operation against the collection in the state it had when we started:

```
// Example 1(b): Using a continuation style,
// and adding resilience to collection changes
//
class LongOperation : public Message {
 Collection myItems;
   int start;
   LongHelper helper;
public:
   LongOperation(
     int start_ = 0,
     LongHelper helper_ = nullptr,
 Collection myItems = nullptr
   )
   : start(start_),
     helper(helper_),
```

```
    myItems(myItems_)
{ }
void run() {
  if( helper == nullptr ) {// if first time through
    helper = GetHelper(); // get helper
myItems = items.copy();// and take a snapshot of items
  }
  int i = 0; // do just another chunk's worth
  for( ; i < ChunkSize && start+i < myItems.size(); ++i ) {
    helper->render( myItems[ start+i ] );
  }
  if( start+i < myItems.size() ) // if not done,
                                 // launch a continuation
    queue.push( LongOperation( start+i, helper, myItems ) );
  else { // if last time through, finish up
    helper->print();
Free( myItems ); // and clean up myItems
  }
 }
}
```

Now the continuation is resilient to the case where other messages may change items, by doing all of its processing using the state the collection had when our operation began.

Note that we have still introduced one other semantic change, in that we're deliberately allowing later messages to run against newer state before this earlier operation on the older state is complete. That's often just fine and dandy, but it's important to be aware that we're buying into those semantics.

All of these considerations apply even more to Option 2. Let's now turn to our second strategy for interleaving work, and see how it offers an alternative set of tradeoffs.

Option 2:Use Reentrant Message Pumping

Option 2 could be subtitled: "Cooperative multitasking to the rescue!" It will be familiar to anyone who's worked on systems based on cooperative multitasking, such as early versions of MacOS and Windows.

Example 2 illustrates the simplest implementation of Option 2, where instead of "saving stack frames on the heap" in the form of continuations, we instead just keep our in-progress state right on the stack where it already is and use nesting (aka reentrancy) to pump additional messages.

```
// Example 2(a): Explicit yield-and-reenter (possibly flawed)
//
class LongOperation : public Message {
public:
  void run() {
    LongHelper helper = GetHelper();
    for( int i = 0; i < items.size(); ++i ) {
if( i % ChunkSize == 0 ) // from time to time
```

```
        Yield();            // explicitly yield control
      helper-> render( items[i] );
    }
    helper->print();
  }
}
```

In a moment we'll consider several options for implementing **Yield**. For now, the key is just that the **Yield** call contains a message pump loop that will process waiting messages, which is what gives them the chance to get unblocked and interleave with this loop.

Option 2 avoids the performance and memory overhead of separate allocation and queueing we saw in Option 1, but it leads to bigger stacks. Stack overflow shouldn't be a problem, however, unless stack frames are large and nesting is pathologically frequent (in which case, there are bigger problems; see next paragraph).

Probably the biggest advantage of this approach is that the code is simpler. We just have to call **Yield** every so often to allow other messages to be processed, and we're golden ... or so we think, because unfortunately it's not actually quite that easy. The code is simpler to write, but requires a little more care to write correctly. Why?

Remember: Beware State Changes When Interleaving, Really

Just as we saw with continuations, so with any other interleaving including **Yield**: Whenever the code **Yields** it must be aware that the thread's state can be changed by the code that executed on this thread during **Yield** operation. It cannot in general assume that any data that is private to the thread, including thread-local variables, remains unchanged across the **Yield** call. With continuations, the issue was a bit easier to remember because that style already requires the programmer to explicitly save the local state off to the side and then return, so that by the time we get to the code where the continuation resumes it's easy to remember that time has passed and the world might have changed.

When using **Yield**-reentrancy instead of continuations, it's easier to forget about this effect, in part because it really is (too) easy to just throw a **Yield** in anywhere. To see how this can cause problems, assume for a moment that semantically we don't care if nested messages change the contents of **items** (which was the case in the discussion around Example 1(b)). That is, assume we don't necessarily need to process a snapshot of the state, but only get through items until we're done. Even with those relaxed requirements, do you notice a subtle bug in Example 2?

Consider: What happens if a nested message changes the size of the **items** collection? If that's possible, then the collection contains at least **i** objects before the **Yield** and the expression **items[i]** is valid, but after the **Yield** the collection may no longer contain **i** objects and so **items[i]** could fail.

In this simple example, we can apply a simple fix. The issue is that we have a window between observing that **i < items.size()** and using **items[i]**, so the simplest fix is to eliminate the problematic window by not yielding in between those points:

```
// Example 2(b): Explicit yield-and-reenter (immediate problem fixed)
//
class LongOperation : public Message {
public:
  void run() {
    LongHelper helper = GetHelper();
    for( int i = 0; i < items.size(); ++i ) {
helper->render( items[i] );
      if( i % ChunkSize == 0 )    // from time to time
```

```
        Yield();                        // explicitly yield control
    }
    helper->print();
  }
}
```

Now the code is resilient to having the **items** collection change during the call.

Of course, as in the discussion around Example 1(b), it's possible that we may not want the collection to change at all, for example to ensure we're processing a consistent snapshot of the collection's state. If so, we may need to save even more off to the side just like we did in Example 1(b), except here it's easier to do because we don't have to mess with a continuation object:

```
// Example 2(c): As resilient as Example 1(b)
//
class LongOperation : public Message {
public:
  void run() {
 Collection myItems = items.copy();
  LongHelper helper = GetHelper();
  for( int i = 0; i < myItems.size(); ++i ) {
    if( i % ChunkSize == 0 )          // from time to time
      Yield();                                    // explicitly yield control
    helper->render(myItems[i] );   // ok now do to this here
  }
  helper->print();
 Free( myItems );
 }
}
```

Notice that we can now correctly use **myItems[i]** both before and after **Yield** because we've insulated ourselves against the problematic state change.

What About *Yield*?

The **Yield** call contains a message pump loop that will process some or all waiting messages. Here's the most straightforward way to implement it:

```
// Example 3(a): A simple Yield that pumps all waiting messages
// (note: contains a subtle issue)
//
void Yield() {
  while( queue.size() > 1 ) { // do message pump
    message = queue.pop();
    message->run();
```

```
  }
}
```

Quick: Is there anything that's potentially problematic about this implementation? Hint: Consider the context of how it's used in Example 2(c) and the order in which messages will be handled.

The potential issue is this: With this **Yield**, Option 2 has a subtle but significant semantic difference from Option 1. In Option 1, the continuation was pushed on the current end of the queue and so enjoyed the fairness guarantee of being executed right after whatever waiting items were in the queue at the time it relinquished control. If more messages arrive while the continuation waits in line, they will get enqueued behind the continuation and be serviced after it in a pure first-in/first-out (FIFO) queue order—modulo priority order if the queue is a priority queue.

Using the **Yield** in Example 3(a), however, we've traded away these pure FIFO queue semantics because we will also execute any new messages that arrive after we call **Yield** but before we completely drain the queue. This might seem innocuous at first; after all, it's usually just "as if" we had called **Yield** slightly later. But notice what happens in the worst case: If messages arrive quickly enough so that the queue never gets completely empty, the operation that yielded might never get control again; it could starve. Starvation is usually unlikely, because normally we don't give a thread more work than it can ever catch up with. But it can arise in practice in systems designed to always keep just enough work ready to keep a thread busy and avoid making it have to wait, and so by design the thread always has a little backlog of work ready to do and the queue stays in a small-but-not-empty steady state. In that kind of situation, beware the possibility of starvation.

The simplest way to prevent this potential problem is to remember how many messages are in the queue and then pump only that many messages:

```
// Example 3(b): A Yield that pumps only the messages
// that were already in the queue at the time it began
//
void Yield() {
 int n = queue.size();
while( n-- > 0 ) {
                // pump 'n' messages
    message = queue.Pop();
    message->run();
  }
}
```

This avoids pumping newer messages that arrive during the **Yield** processing and usually guarantees progress (non-starvation) for the function that called **Yield**, as long as we avoid pathological cases where the nested messages **Yield** again. (Exercise for the reader: Instrument **Yield** to detect starvation due to nesting.)

Incidentally, note that once again we're applying the technique of taking a snapshot of the state of the system as it was when we began the operation, just like we did in Examples 1(b) and 2(c) where we took a copy of the items collection. In this case, thanks to the FIFO nature of the queue, we don't need to physically copy the queue; it's sufficient to remember just the number of items to pump.

Summary

Sometimes a thread has to interleave its work in order to stay responsive, and avoid blocking new time-sensitive requests that may arrive while it's

already processing a longer but lower-priority operation.

There are two main ways to interleave: Option 1 is to use continuations, which means saving the operation's intermediate local state in an object on the heap and creating a new message containing "the rest of the work left to do," which gets inserted into the queue so that we can handle other messages and then pick up and resume the original work where we left off. Option 2 is to use cooperative multitasking and reentrancy by yielding to a function that will pump waiting messages; this yields simpler code and avoids some allocation and queueing overhead because the locals can stay on the stack where they already are, but it also allows deeper stack growth.

In both cases, remember: The issues of interleaving other work are really nasty and it's all too easy to get things wrong, especially when **Yield** calls are sprinkled around and make the program very hard to reason about locally. Always be careful to remember that the interleaved work could have side effects, and make sure the longer work is resilient to changes to the data it cares about. If we don't do that correctly and consistently, we'll expose ourselves to a class of subtle and timing-dependent surprises. Next month, we'll consider a way to avoid this class of problems by making sure the state of the system is valid at all times, even with partially done work outstanding. Stay tuned.

Notes

[1] H. Sutter. "Use Threads Correctly = Isolation + Asynchronous Messaging (Dr. Dobb's Digest, March 2009).

Acknowledgments

Thanks to Terry Crowley for his insightful comments on drafts of this article.